

Topic 3.5

3.5: Inline Functions

Generally, the compiled body of a function occupies a significant amount of space. Wherever a function is invoked, there is a special sequence of instructions that is executed to effect the function invocation. The number of instructions in this sequence is dependent on the actual computer hardware, and there may be from only a few instructions to perhaps many. Even if the number of instructions is small, they tend to be time consuming. Thus, function invocations are expensive in terms of execution time.

This is a time/space tradeoff. The code is smaller, but perhaps less efficient timewise than it could be if we didn't use functions. Usually this is OK.

In some situations, execution time is critical. In other situations, the function body is actually smaller than the code necessary to invoke it. In these situations, we can *use inline functions*.

Within a class declaration, if the body of a member function is specified, that member function is implicitly inline. Here is an example:

```
class Account
{
    ...
    /* No-argument constructor with body
       specified causes this function to be
       implicitly inline. */
    Account( void ) {initialized = 0;}
    ...
}
```

Functions declared **inline** are explicitly inline. Do not use **inline** cavalierly—it can result in *enormous* executables.

From a code maintenance perspective, use of inline functions may result in confusion because you would now have to look in two places, the header file and the implementation file, to find all the function definitions for a specified class, etc.

The following is an example that illustrates the use of an **inline** function:

```
// FILENAME:   tripleit.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ Express Edition
// REQUIRED:    tripleit.cpp
```

```
// PURPOSE:  
//   Multiple numbers by 3.  
//   Illustrate inline directive.  
  
#include <iostream>  
  
using namespace std;  
  
inline void triple_it( int &x )  
{  
    x *= 3;  
}  
  
int main(void)  
{  
    int y=17;  
  
    cout << "Originally y=" << y << endl;  
    triple_it(y);  
    cout << "After tripling, y=" << y << endl;  
  
    return 0;  
}
```

Output from tripleit.cpp:

```
C:\DATA\TEACHING\UCB\CPP> tripleit  
Originally y=17  
After tripling, y=51
```

Topic 3.6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-3-dot-6>)